Documentation des choix - Projet POO

Alexandre, Samir, Natacha et Louis

Package `balls`:

Pour le package `balls`, nous avons suivi les directives données dans le TP et avons procédé à quelques spécifications :

Classe `Ball`:

Nous avons choisi de faire hériter la classe `Ball` de `Point` pour qu'on puisse réutiliser les méthodes et attributs déjà fournis par la classe `Point`. Cela évite de dupliquer du code et rend la gestion de la position des balles plus simple et cohérente.

Classe `Balls` :

Nous avons choisi de faire hériter la classe `Balls` de `ArrayList<Ball>`. Ainsi, `Balls` devient une liste spécialisée de `Ball`, ce qui permet de regrouper et gérer l'ensemble des balles directement au sein de cette classe.

Classe `Config`:

Nous avons choisi d'ajouter une classe `Config` dans chaque package afin de centraliser les paramètres de configuration. Dans le package `balls`, cette classe permet par exemple de modifier la taille de l'écran. Cela rend les réglages plus accessibles et évite la dispersion des constantes dans le code.

Classe `BallEvent` :

Cet événement déclenche le calcul de la nouvelle position de toutes les balles. Il se replanifie à l'instant suivant pour assurer la continuité de l'animation.

Package `automatons`:

Nous avons choisi de regrouper toute la gestion des automates cellulaires dans le package `automatons`, ce qui facilite la compréhension et permet de mieux distinguer les différentes parties du projet.

Package `grid`:

Nous avons décidé de centraliser la gestion de la grille en créant un package `grid`. C'est uniquement dans ce package que l'on manipule des objets de type `Grid`. Ce choix de conception a été pensé pour rendre le code plus générique : par exemple, si l'on souhaitait utiliser un type différent de `Grid`, cela serait plus facile à intégrer.

Dans les différents jeux — **Conway**, **Immigration** et **Schelling** — nous n'utilisons que des `Cell`. L'initialisation et l'utilisation des `Grid` sont donc entièrement gérées dans le package `grid`.

Ce choix nous permet d'éviter la duplication du code lié aux grilles, comme leur création ou leur manipulation.
Pour les trois jeux, la gestion de la grille est identique : seules les règles propres à chaque jeu diffèrent.

C'est pour cela que nous avons décidé de créer la classe `EvolutionRules`.

Classe `EvolutionRules`:

La classe `EvolutionRules` est une classe abstraite qui centralise les différentes fonctions qui seront utilisées par les différents jeux, par exemple la fonction `computeNextCellState` est une fonction générique dont l'implémentation varie selon le jeu considéré. Chaque type de jeu (Conway, Immigration, Schelling) fournit donc sa propre version de cette méthode pour définir ses règles d'évolution spécifiques.

Classe `Cell`:

Nous avons choisi de centraliser les méthodes et les attributs dans la classe `Cell`, ce qui permet de gérer individuellement les cellules d'une grille. Cette classe regroupe les informations et opérations propres à une cellule, notamment les attributs définissant son état ainsi que les méthodes permettant de le modifier.

Classe `Grid`:

La classe `Grid` implémente une matrice circulaire de `Cell`. Elle centralise la gestion de toutes les cellules du système : création, accès, réinitialisation et récupération des voisins.
Elle stocke également l'état initial de chaque cellule afin de pouvoir réinitialiser la grille à tout moment. La grille est circulaire, ce qui signifie que les cellules situées sur un bord ont pour voisins des cellules situées sur le bord opposé.

Classe `GridSimulator`:

La classe `GridSimulator` implémente l'interface `Simulable`, ce qui lui permet d'afficher la grille, de la réinitialiser et de faire évoluer les états des cellules. Elle centralise l'exécution de la simulation en appliquant les règles d'évolution fournies par un objet `EvolutionRules`. Le simulateur se charge de l'affichage graphique des cellules.

Package `conway`, `immigration` et `schelling`:

Les trois packages correspondant aux jeux sont structurés de manière similaire.
Chacun contient une classe `Config`, comme expliqué précédemment, permettant de centraliser les paramètres propres au jeu.
Chaque package possède également une classe de test qui instancie l'objet rules approprié (`ConwayRules`, `ImmigrationRules` ou `SchellingRules`), initialise l'interface graphique ainsi que la grille, puis lance la simulation.

Classe `ConwayRules`, `ImmigrationRules` et `SchellingRules`:

Les trois classes fonctionnent de la même manière : elles héritent toutes de la classe abstraite `EvolutionRules`, comme expliqué précédemment, et implémentent les méthodes `reset`, `computeNextCellState` et `generateNewStateValues`.
Nous avons choisi de regrouper ces méthodes et ces classes car la seule différence entre les trois règles réside dans leur implémentation. C'est pourquoi nous avons estimé que cette architecture était adaptée et cohérente.

Package `boids`:

Classe `Vector`:

Afin d'implémenter et de gérer plus facilement les boids on a décidé de modéliser la position, la vitesse, l'accélération et les différentes forces appliquées aux boids par des vecteurs. La classe `Vector` possède 2 attributs : les coordonnées x et y représentées par des *doubles* pour plus de précision lors des calculs. Lorsque que le vecteur représente la position, celui-ci est considéré comme un vecteur partant de (0,0). Sinon c'est un vecteur partant de la position actuelle du boid. Cette classe implémente toutes les méthodes nécessaires pour effectuer des calculs sur les vecteurs.

Classe `Boid`:

Cette classe représente un boid individuel. Chaque Boid possède comme attribut l'ensemble (HashSet) de ses voisins, l'ensemble (HashSet) de ses "amis" (ses voisins du même groupe), un vecteur accélération ,vitesse et position. À chaque étape, on actualise la position du boid en appelant la méthode **update** qui effectue plusieurs actions :

On actualise d'abord l'ensemble de voisins et l'ensemble d'amis du boid en appelant la méthode **updateNeighbors**. Après ce prétraitement, on calcule les forces de cohésion, alignement, séparation et bordure (cette dernière permet de s'éloigner des murs)

Les forces de cohésion et d'alignement prennent en compte uniquement les **amis** dans le cercle de rayon *boids.neighborDistance* tandis que la force de séparation prend en compte tous les **voisins** dans le cercle de rayon *boids.separationFieldRadius*. On différencie ces 2 cercles car on voulait avoir la possibilité de ne pas appliquer la force de séparation de façon aussi étendue que les autres forces.

Toutes les forces considèrent aussi uniquement les boids présents dans leur champ de vision (défini par *boids.visionField*). Le calcul de ces forces passe par l'appel de 4 méthodes spécifiques. On applique ensuite une pondération (dépendante de chaque groupe de boids) sur les 3 premières forces pour faire en sorte que certaines forces aient plus d'influence que d'autres. On applique ensuite chacune de ces forces au boid en utilisant la méthode **applyForce** qui va les ajouter à l'accélération. On ajoute ensuite l'accélération à la vitesse actuelle, vitesse dont on force ensuite la magnitude à la valeur *maxSpeed* afin que tous les boids aient une vitesse constante (ici c'est donc uniquement la direction qu'on affecte). On ajoute ensuite cette vitesse à la position actuelle et on réinitialise l'accélération à 0.

Classe `Boids`:

Cette classe permet de représenter un groupe de boids. Elle étend la classe *HashSet<Boid>* afin de stocker l'ensemble des boids de ce groupe (on utilise un set car pas besoin d'ordre spécifique parmis les boids). Chaque groupe de boids dépend de plusieurs paramètres qui changent leurs comportements. Ces paramètres sont la distance maximale d'un voisin, le champ de vision, la vitesse maximum, la puissance maximum d'une force, la couleur des boids, les multiplicateurs de chaque force (pour la pondération), le rayon de séparation, la taille des boids, le délai pour la planification et l'ensemble de tous les groupes (afin de pouvoir aussi éviter les boids qui ne sont pas dans le groupe).

Classe `Triangle`:

Cette classe implémente la classe *GraphicalElement* de Java. Elle permet de dessiner un triangle lorsqu'on appelle gui.addGraphicalElement pour dessiner un boid. Cela nous permet d'avoir des boids visuellement parlants, chaque triangle étant dessiné dans la direction de son boid respectif.

Classe `Fish`:

Cette classe, similaire à Triangle, permet de dessiner un poisson plutôt qu'un triangle. Le choix entre les deux formes se fait dans le fichier de configuration du package *boids*.

Classe `BoidsGroupEvent` :

Cette classe hérite de Event. Son rôle est de déclencher la mise à jour d'un groupe de boids spécifique. Dans sa méthode *execute()*, elle parcourt tous les boids du groupe et appelle leur méthode *update()*. Après son exécution, elle se replanifie elle-même pour une date future en créant un nouvel événement et en l'ajoutant à EventManage. C'est pour assurer la continuité de la simulation.

Package `events`:

Classe `Event` :

Une classe abstraite qui représente une action planifiable. Le fait qu'elle implémente Comparable<Event> permet de déléguer le tri des événements par date à une structure de données.

Classe `EventManager` :

Le choix d'utiliser une PriorityQueue permet de trier une liste à chaque pas de temps, la PriorityQueue maintient les événements ordonnés à chaque insertion, garantissant que l'événement le plus proche dans le temps est toujours accessible en temps quasi constant. Quand sa méthode next() est appelée, il exécute tous les événements dont l'heure est venue.

Package `test`:

Les tests du projet sont organisés dans un package séparé, parallèle au code source : le code métier se trouve dans src/main/... et les tests dans src/test/..., en conservant la même structure de sous-packages (par exemple grid/Cell.java et grid/CellTest.java).
Cette organisation permet de bien séparer la logique de l'application et les tests, et de voir rapidement quelles classes sont couvertes par des tests.
Pour les tests, nous avons utilisé la bibliothèque JUnit : chaque classe de test contient plusieurs méthodes annotées @Test qui s'appuient sur des assertions (assertEquals, assertTrue, etc.) pour vérifier le bon comportement des différentes règles de simulation (positions valides, respect des règles de déplacement, cases libres, etc.).

Explication de certains tests important :

Classe `GridTest`:

testGetCellNeighborsOrderAndContents vérifie que la méthode chargée de récupérer les voisins d'une cellule renvoie bien les bonnes cellules *dans le bon ordre*, ce qui est indispensable pour garantir une évolution cohérente et déterministe de l'automate. De son côté, **testReinitRestoresInitialStates** s'assure que la méthode de réinitialisation restaure correctement l'état initial de la grille : après un appel à reinit(), toutes les cellules doivent retrouver exactement les valeurs qu'elles avaient au début de la simulation.

Les trois batteries de tests – `ImmigrationEvolutionRulesTest`, `SchellingRuleTest` et `ConwayRulesTest` – vérifient chacune le bon fonctionnement des règles d'évolution propres à leur automate cellulaire. Malgré les différences de logique entre les trois jeux, les tests suivent une structure similaire : validation du calcul de l'état suivant, cohérence de l'évolution, et contrôle de la génération des états initiaux.